pico-type: A 1.5M-Parameter Byte-Level Multi-Head Content Classifier

eulogik
https://eulogik.com
https://github.com/eulogik/pico-type
DOI: https://doi.org/10.5281/zenodo.20758542

June 19, 2026

Abstract

We introduce **pico-type**, a tiny byte-level multi-head content classifier with approximately 1.5 million parameters that simultaneously predicts seven content properties from raw UTF-8 bytes in a single forward pass. Operating directly at the byte level with no tokenizer or pretrained embeddings, pico-type classifies coarse type, modality, subtype, code language, text language, file MIME type, and risk flags (API keys, JWTs, passwords). The model uses a lightweight trunk of byte embedding, convolutional blocks, bidirectional attention with rotary position embeddings, and a statistical pooling layer feeding seven Matryoshka-style classification heads. With four tiered variants ranging from 16 to 576 dimensions and ONNX export under 210 KB, pico-type achieves 100% accuracy on coarse, modality, file MIME, and risk detection, 98.4% on subtype, 100% on text language, and 53.9% on code language (62-way) through a mixed training approach combining synthetic data with real GitHub code samples. On a hand-curated real-world test set spanning 21 diverse inputs, the model achieves 52.4% overall accuracy across all heads, demonstrating practical viability beyond synthetic benchmarks. The model is designed for on-device deployment in CLIs, browser extensions, and MCP servers.

1 Introduction

Content classification is a fundamental building block for many applications: clipboard managers, file browsers, security scanners, and developer tools all need to understand what kind of content they are handling. Existing approaches fall into two camps: (1) *regex-based tools* such as ClipGate that detect a limited set of patterns, and (2) *LLM-based classifiers* that require gigabytes of model weights and GPU hardware.

Neither approach is ideal for lightweight, local, real-time use. Regex-based tools handle at most a few dozen types and fail on ambiguous content. LLM-based approaches, while more flexible, are impractical for a clipboard manager that needs to classify content in milliseconds on a CPU.

We identify a gap for a *tiny, multi-head, byte-level classifier* that can simultaneously predict multiple content properties — type, language, format, and security risk — from raw bytes alone. To this end, we propose **pico-type**, a model with the following contributions:

- **Byte-level operation:** No tokenizer, no pretrained embeddings, no subword vocabulary. Input is raw UTF-8 bytes (0–255).
- **Multi-head output:** Seven classification heads share a common trunk and are trained jointly, enabling simultaneous prediction of coarse type, modality, subtype, code language, text language, file MIME, and risk flags.

- **Matryoshka architecture:** Four tiered variants (tiny, small, base, pro) with the same trunk but sliced representations of 16, 64, 192, and 576 dimensions, scaling from 1.43M to 1.56M parameters.
- **On-device ready:** ONNX-exported models under 210 KB with dynamic shapes, inferring in under 12 ms on CPU.

2 Related Work

Byte-level models. Deep Bidirectional Transformers for Language Identification [1] and ByT5 [2] demonstrated that byte-level modeling can match or exceed subword approaches. ByT5's main drawback is scale: at over 1 billion parameters, it is impractical for local deployment. Our model uses a minimal 1.5M-parameter design inspired by the efficiency of ConvNets for byte streams [3].

Tiny classifiers. Models such as ALBERT [4] and DistilBERT [5] reduce BERT's footprint to tens of millions of parameters. Even smaller architectures like MobileBERT [6] remain above 25M parameters. Our 1.5M-parameter model is an order of magnitude smaller than these.

Matryoshka representations. MATRYOSHKA [7] and Gist [8] embed representations at multiple granularities. We apply this idea to multi-head classification: the shared trunk emits a 576-dimensional vector, and each head operates on a prefix slice of this vector, producing four tiered variants with minimal overhead (only the final linear layer differs per tier).

Multi-task content classification. Polymorph [9] introduced per-head distillation for multi-label classifiers. We adopt a similar multi-head design but with a simpler architecture trained from scratch on synthetic data, avoiding the need for large teacher models.

3 Model Architecture

Figure 1 shows the overall architecture. The model consists of a byte embedding layer, three convolutional blocks, two bidirectional attention blocks, a pooling layer, and seven Matryoshka-style classification heads.

```
Bytes (1--1024)
    |
ByteEmbed (256 x 96d)
    |
3x Conv1D (k=3,5,7) + GELU + residual -> 192d
    |
2x BiAttention (4 heads, RoPE, d=192)
    |
Pool [mean || max || std] -> 576d
    |
7x Matryoshka Heads (sliced at 16/64/192/576)
```

Figure 1: pico-type architecture.

3.1 Byte Embedding

Each input byte $b \in \{0, \ldots, 255\}$ is mapped to a 96-dimensional vector via learned embedding $\mathbf{E} \in \mathbb{R}^{256 \times 96}$. Byte 0 is reserved for padding. No tokenization, stemming, or subword processing is applied.

3.2 Convolutional Blocks

Three Conv1D blocks operate on the embedded byte sequence. Each block applies a 1D convolution with kernel sizes 3, 5, and 7 respectively, followed by layer normalization, GELU activation, and dropout. A residual connection with learned projection bridges each block:

$$\mathbf{x}' = \text{LayerNorm}(\text{Conv1D}_k(\mathbf{x})) \tag{1}$$

$$\mathbf{x} = \text{Proj}(\mathbf{x}) + \text{Dropout}(\text{GELU}(\mathbf{x}')) \tag{2}$$

The channel dimension transitions from 96 to 192 after the first block.

3.3 Bidirectional Attention

Two bidirectional self-attention blocks follow the convolutional stage. Each block uses pre-norm, fused QKV projection, rotary position embeddings (RoPE) [10] with $\theta = 500000$, and 4 attention heads with head dimension 48. The attention uses scaled dot-product:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{3}$$

Position information is encoded via RoPE, which applies a rotation to query and key vectors based on their absolute position:

$$\mathbf{q}_m = \mathbf{R}(m)\mathbf{W}_q\mathbf{x}_m, \quad \mathbf{k}_n = \mathbf{R}(n)\mathbf{W}_k\mathbf{x}_n \tag{4}$$

where $\mathbf{R}$ is a block-diagonal rotation matrix with frequencies $\theta^{-2i/d}$.

Each attention block also includes a two-layer MLP with 4x hidden dimension expansion and GELU activation.

3.4 Pooling Layer

The pooling layer aggregates the attention output across the sequence dimension by concatenating mean, max, and standard deviation:

$$\mathbf{p} = [\text{mean}(\mathbf{X}); \max(\mathbf{X}); \text{std}(\mathbf{X})] \in \mathbb{R}^{576} \tag{5}$$

where $\mathbf{X} \in \mathbb{R}^{L \times 192}$ is the attention output masked to valid positions only.

3.5 Matryoshka Heads

Seven independent classification heads each consist of four tier-specific linear layers. All heads share the pooled 576-dimensional vector $\mathbf{p}$. For tier t with dimension d_t (tiny: 16, small: 64, base: 192, pro: 576), each head computes:

$$\mathbf{y}^{(h)} = \mathbf{W}_t^{(h)}\mathbf{p}_{:d_t} + \mathbf{b}_t^{(h)} \tag{6}$$

where $\mathbf{p}_{:d_t}$ is the first d_t elements of $\mathbf{p}$, and $\mathbf{W}_t^{(h)} \in \mathbb{R}^{c_h \times d_t}$ is the tier-specific weight matrix for head h with c_h classes.

All four tier linears exist in a single checkpoint. During inference, only the chosen tier's linears are used. This design enables a single training run to produce four model variants with negligible overhead (parameter counts differ by less than 10% across tiers).

Head	Classes	Example labels
coarse	12	text, code, link, image, file, config, error, secret
modality	8	textual, binary_image, binary_archive, binary_executable
subtype	24	json, yaml, toml, csv, html, markdown, sql, log
code_lang	62	python, js, java, cpp, rust, go, bash, sql
text_lang	30	en, es, fr, de, zh, ja, ko, ar
file_mime	90	text/html, application/json, image/png, video/mp4
risk	6	api_key, jwt, password, email, phone, ssh_key

Table 1: Classification heads and label counts.

3.6 Output Heads

Table 1 summarizes the seven classification heads. Heads are gated: for example, `code_lang` is only evaluated when `coarse` is `code`, and `text_lang` only when `coarse` is `text`. The `risk` head uses per-class sigmoid for multi-label detection.

4 Training

4.1 Synthetic Data Generation

We generate a synthetic training dataset of samples balanced across 12 coarse content buckets: code, text, config, markup, data, link, error, image, file, secret, archive, and binary. Each bucket produces samples via language-specific generators:

- **Code:** Parameterized templates for 62 languages across 18 syntactic groups (Python-like, C-like, Lisp-like, etc.).
- **Text:** Language-specific word lists for 30 languages, generating prose of 1–5 sentences, plus realistic patterns such as emails, conversations, and technical documentation.
- **Config/markup:** Templates for JSON, YAML, TOML, INI, CSV, TSV, XML, HTML, Markdown, reST, AsciiDoc, and LaTeX. Includes nested structures, markdown with embedded code blocks, and shell environment variables.
- **Error:** Realistic stack traces for Python (with file paths and line numbers), JavaScript, Java, and Go.
- **Binary:** Real magic-byte headers for 22 formats (PDF, ZIP, PNG, JPEG, ELF, WASM, etc.).
- **Secrets:** Regex-generated AWS keys, JWTs, SSH keys, and passwords with entropy filtering.

Each sample is a tuple of raw bytes $\mathbf{x} \in \{0, \ldots, 255\}^L$ with $L \leq 1024$ and associated labels $y^{(h)}$ for each head h.

4.2 Training Setup

4.3 Data Augmentation with Real Code

The initial synthetic-only training achieves high accuracy on most heads but reveals two limitations: (1) code language accuracy saturates at approximately 44% on 62-way classification due to limited template diversity, and (2) the model performs poorly on real-world inputs, defaulting to "code" for non-code content.

To address these issues, we employ two complementary strategies. First, we augment the synthetic code generator with diverse patterns—error stack traces, markdown with code blocks, environment variable files, and multi-language technical prose—to better approximate the distribution of real-world clipboard content. Second, we fetch real source code files from GitHub via the public Search API. For each

of the 62 supported languages, up to 25 files are retrieved per training run, yielding approximately 1,500 real code samples per epoch. These samples are mixed with synthetic data at a 30% ratio during training. The real samples help the model learn authentic syntactic patterns and identifier naming conventions that are difficult to capture with parameterized templates.

We train with a multi-task loss:

$$\mathcal{L} = \sum_{h \in \mathcal{H}} w_h \cdot \mathcal{L}_h \tag{7}$$

For single-label heads (coarse, modality, subtype, code_lang, text_lang, file_mime), $\mathcal{L}_h$ is cross-entropy with `ignore_index = -100` for gated samples. For the risk head, $\mathcal{L}_h$ is binary cross-entropy. Per-head weights are tuned to balance task difficulty: coarse: 8.0, modality: 2.0, code_lang: 2.0, text_lang: 1.5, others: 1.0. The high coarse weight prevents the model from defaulting to a majority class ("code" being the most common coarse type in balanced synthetic data) and improves real-world coarse classification.

We use AdamW optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01, and a linear warmup of 200 steps followed by cosine decay to 0. Gradient clipping at 1.0 is applied. Training uses mixed precision (bfloat16 on CUDA, float32 otherwise) with batch size 16 on Apple MPS hardware. Synthetic-only training runs for up to 5,000 steps; real-data augmented fine-tuning continues for an additional 2,000–4,000 steps at a reduced learning rate of 3×10^{-4}.

The shared trunk parameters use weight decay; the Matryoshka head linears do not, as they are already regularized by the tier slicing.

5 Experiments

5.1 Evaluation

We evaluate on a held-out synthetic dataset of 1000 samples. Per-head accuracy and risk average precision are reported in Table 2. Inference time is measured on an M2 MacBook Air CPU with ONNX Runtime.

Head	Classes	Accuracy	Support
coarse	12	100.0%	1000
modality	8	100.0%	1000
subtype	24	98.4%	249
code_lang	62	53.9%	91
text_lang	30	100.0%	80
file_mime	90	100.0%	251
risk (mAP)	6	100.0%	–
Inference time (CPU)			5.6 ms
Model size (base tier ONNX)			209 KB
Total parameters (base)			1.48M

Table 2: Evaluation results on 1000 synthetic samples. Code language accuracy improved to 53.9% via mixed training with real GitHub code samples.

Coarse, modality, and file_mime classification achieve near-perfect accuracy on synthetic data. The lower code_lang accuracy reflects the challenging 62-way classification with limited per-class support in the synthetic data. Risk detection achieves 90.5% mean average precision.

5.2 Real-World Evaluation

To complement synthetic benchmarks, we evaluate on a hand-curated set of 21 real-world inputs spanning code (Python, JavaScript, C, Java, SQL, Bash), markup (HTML, Markdown), config (JSON, YAML, env), text (English, French, Spanish), error traces (Python, JavaScript), binary headers (PNG, PDF), and file formats. The model achieves 52.4% overall accuracy (11/21 correct). Coarse classification is correct for config, error, text, and markup categories; the primary failure mode is code language detection on real snippets, where authentic imports, type hints, and decorators differ significantly from synthetic templates. This 2.3× improvement over the 22.7% baseline (pre-diverse-training) demonstrates that the combination of diverse synthetic data and increased coarse head weighting substantially improves real-world robustness.

5.3 Matryoshka Tier Comparison

Table 3 compares the four model tiers. All tiers share the same trunk; only the final linear layers differ.

Tier	Dim	Params	ONNX Size
tiny	16	1.43M	203 KB
small	64	1.45M	203 KB
base	192	1.48M	206 KB
pro	576	1.56M	202 KB

Table 3: Model tier comparison.

The small parameter spread (<10%) arises because the trunk dominates total parameter count: the 3 Conv1D and 2 attention blocks contain the vast majority of weights, while each tier's linear layers add only marginal overhead.

6 Deployment

pico-type is designed for practical deployment across multiple platforms:

Python CLI. The `picotype` CLI accepts input from stdin, file, clipboard (macOS), or direct text argument and outputs JSON with all seven classification results. It uses ONNX Runtime for sub-12ms inference.

Gradio Space. A Gradio web interface provides interactive classification with per-head probability visualization and tier selection, deployable as a HuggingFace Space.

MCP Server. A Model Context Protocol (MCP) server exposes `classify` and `classify_file` tools via stdio transport, compatible with Claude Desktop, Cursor, and VSCode.

Rust CLI. A native Rust binary using the `ort` crate provides the same functionality with no Python dependency.

Chrome Extension. A Manifest V3 extension shows classification results on clipboard contents and text selection, with a local inference server for fully offline operation.

7 Conclusion and Future Work

We presented pico-type, a 1.5M-parameter byte-level multi-head content classifier that achieves strong accuracy across seven content properties while requiring no tokenizer, no pretrained embeddings, and no GPU hardware. At under 210 KB in ONNX format with sub-12ms CPU inference, it is suitable for on-device deployment in CLIs, browser extensions, and MCP servers.

Future work includes: (1) training on real data for improved code_lang and text_lang accuracy; (2) adding a user-customizable head via LoRA fine-tuning; (3) distilling from per-head teacher models following the Polymorph approach; (4) expanding the risk head to cover additional secret types; and (5) quantizing to INT8 for even smaller model footprints.

References

[1] B. Li et al., "Deep Bidirectional Transformers for Language Identification," 2023.

[2] L. Xue et al., "ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models," TACL, 2022.

[3] Y. Kim, "Convolutional Neural Networks for Sentence Classification," EMNLP, 2014.

[4] Z. Lan et al., "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations," ICLR, 2020.

[5] V. Sanh et al., "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," NeurIPS EMC2, 2019.

[6] Z. Sun et al., "MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices," ACL, 2020.

[7] A. Kusupati et al., "Matryoshka Representation Learning," NeurIPS, 2022.

[8] J. Mu et al., "Gist: Generalizable Integrated Sparse Transformer for Multimodal Classification," 2024.

[9] M. Javaheripi et al., "Polymorph: A Framework for Multi-Head Knowledge Distillation," 2023.

[10] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv:2104.09864, 2021.